

11

MLOps and LLMOps

Throughout the book, we've already used **machine learning operations (MLOps)** components and principles such as a model registry to share and version our fine-tuned **large language models (LLMs)**, a logical feature store for our fine-tuning and RAG data, and an orchestrator to glue all our ML pipelines together. But MLOps is not just about these components; it takes an ML application to the next level by automating data collection, training, testing, and deployment. Thus, the end goal of MLOps is to automate as much as possible and let users focus on the most critical decisions, such as when a change in distribution is detected and a decision must be taken on whether it is essential to retrain the model or not. But what about **LLM operations (LLMOps)**? How does it differ from MLOps?

The term *LLMOps* is a product of the widespread adoption of LLMs. It is built on top of MLOps, which is built on top of **development operations (DevOps)**. Thus, to fully understand what LLMOps is about, we must provide a historical context, starting with DevOps and building on the term from there—which is precisely what this chapter will do. At its core, LLMOps focuses on problems specific to LLMs, such as prompt monitoring and versioning, input and output guardrails to prevent toxic behavior, and feedback loops to gather fine-tuning data. It also focuses on scaling issues that appear when working with LLMs, such as collecting trillions of tokens for training datasets, training models on massive GPU clusters, and reducing infrastructure costs. Fortunately for the common folk, these issues are solved mainly by a few companies that fine-tune foundational models, such as Meta, which provides the Llama family of models. Most companies will adopt these pre-trained foundational models for their use cases, focusing on LLMOps problems such as prompt monitoring and versioning.

On the implementation side of things, to add LLMOps to our LLM Twin use case, we will deploy all our ZenML pipelines to AWS. We will implement a **continuous integration and continuous deployment (CI/CD)** pipeline to test the integrity of our code and automate the deployment process, a **continuous training (CT)** pipeline to automate our training, and a monitoring pipeline to track all our prompts and generated answers. This is a natural progression in any ML project, regardless of whether you use LLMs.

In previous chapters, you learned how to build an LLM application. Now, it's time to explore three main goals related to LLMOps. The first one is to gain a theoretical understanding of LLMOps, starting with DevOps, then moving to the fundamental principles of MLOps, and finally, digging into LLMOps. We don't aim to provide the whole theory on DevOps, MLOps, and LLMOps, as you could easily write an entire book on these topics. However, we want to build a strong understanding of why we make certain decisions when implementing the LLM Twin use case.

Our second goal is to deploy the ZenML pipelines to AWS (currently, we've deployed only our inference pipeline to AWS in *Chapter 10*). This section will be hands-on, showing you how to leverage ZenML to deploy everything to AWS. We need this to implement our third and last goal, which is to apply what we've learned in the theory section to our LLM Twin use case. We will implement a CI/CD pipeline using GitHub Actions, a CT and alerting pipeline using ZenML, and a monitoring pipeline using Opik from Comet ML.

Thus, in this chapter, we will cover the following topics:

- The path to LLMOps: Understanding its roots in DevOps and MLOps
- Deploying the LLM Twin's pipelines to the cloud
- Adding LLMOps to the LLM Twin

The path to LLMOps: Understanding its roots in DevOps and MLOps

To understand LLMOps, we have to start with the field's beginning, which is DevOps, as it inherits most of its fundamental principles from there.

Then, we will move to MLOps to understand how the DevOps domain was adapted to support ML systems. Finally, we will explain what LLMOps is and how it emerged from MLOps after the widespread adoption of LLMs.

DevOps

Manually shipping software is time-consuming, error-prone, involves security risks, and doesn't scale. Thus, DevOps was born to automate the process of shipping software at scale. More specifically, DevOps is used in software development, where you want to completely automate your building, testing, deploying, and monitoring components. It is a methodology designed to shorten the development lifecycle and ensure continuous delivery of high-quality software. It encourages collaboration, automates processes, integrates workflows, and implements rapid feedback loops. These elements contribute to a culture where building, testing, and re-releasing software becomes more reliable and faster.

Embracing a DevOps culture offers significant advantages to an organization, primarily boosting operational efficiency, speeding up feature delivery, and enhancing product quality. Some of the main benefits include:

- **Improved collaboration:** DevOps is pivotal in creating a more unified working environment. Eliminating the barriers between development and operations teams fosters enhanced communication and teamwork, leading to a more efficient and productive workplace.
- **Boosted efficiency:** Automating the software development lifecycle reduces manual tasks, errors, and delivery times.
- **Ongoing improvement:** DevOps is not just about internal processes. It's about ensuring that the software effectively meets user needs. Promoting a culture of continuous feedback enables teams to quickly adapt and enhance their processes, thereby delivering software that genuinely satisfies the end users.
- **Superior quality and security:** DevOps ensures swift software development while maintaining high quality and security standards through CI/CD and proactive security measures.

The DevOps lifecycle

As illustrated in *Figure 11.1*, the DevOps lifecycle encompasses the entire journey from the inception of software development to its delivery, up-keep, and security. The key stages of this lifecycle are:

1. **Plan:** Organize and prioritize the tasks, ensuring each is tracked to completion.
2. **Code:** Collaborate with your team to write, design, develop, and securely manage code and project data.
3. **Build:** Package your applications and dependencies into an executable format.
4. **Test:** This stage is crucial. It's where you confirm that your code functions correctly and meets quality standards, ideally through automated testing.
5. **Release:** If the tests pass, flag the tested build as a new release, which is now ready to be shipped.
6. **Deploy:** Deploy the latest release to the end users.
7. **Operate:** Manage and maintain the infrastructure on which the software runs effectively once it is live. This involves scaling, security, data management, and backup and recovery.
8. **Monitor:** Track performance metrics and errors to reduce the severity and frequency of incidents.



Figure 11.1: DevOps lifecycle steps

The core DevOps concepts

DevOps encompasses various practices throughout the application lifecycle, but the core ones that we will touch on throughout this book are:

- **Deployment environments:** To thoroughly test your code before shipping it to production, you must define multiple pre-production environments that mimic the production one. The most common approach is to create a dev environment where the developers can test their latest features. Then, you have a staging environment where the QA team and stakeholders tinker with the application to find bugs and experience the latest features before they ship to the users. Lastly, we have the production environment, which is exposed to end users.
- **Version control:** Used to track, manage, and version every change made to the source code. This allows you to have complete control over the evolution of the code and deployment processes. For example, without versioning, tracking changes between the dev, staging, and production environments would be impossible. By versioning your software, you always know what version is stable and ready to be shipped.

- **Continuous integration (CI):** Before pushing the code into the dev, staging, and production main branches, you automatically build your application and run automated tests on each change. After all the automated tests pass, the feature branch can be merged into the main one.
- **Continuous delivery (CD):** Continuous delivery works in conjunction with CI and automates the infrastructure provisioning and application deployment steps. For example, after the code is merged into the staging environment, the application with the latest changes will be automatically deployed on top of your staging infrastructure. After, the QA team (or stakeholders) starts manually testing the latest features to verify that they work as expected. These two steps are commonly referred to together as CI/CD.

Note that DevOps suggests a set of core principles that are platform/tool agnostic. However, within our LLM Twin use case, we will add a version control layer using GitHub, which aims to track the evolution of the code. Another popular tool for version control is GitLab. To implement the CI/CD pipeline, we will leverage the GitHub ecosystem and GitHub Actions, which are free for open-source projects. Other tool choices are GitLab CI/CD, CircleCI, and Jenkins. Usually, you pick the DevOps tool based on your development environment, customization, and privacy needs. For example, Jenkins is an open-source DevOps tool you can host yourself and control fully. The downside is that you must host and maintain it yourself, adding a complexity layer. Thus, many companies choose what works best with their version control ecosystem, such as GitHub Actions or GitLab CI/CD.

Now that we've established a solid understanding of DevOps, let's explore how the MLOps field has emerged to keep these same core principles in the AI/ML world.

MLOps

As you might have worked out by now, MLOps tries to apply the DevOps principles to ML. The core issue is that an ML application has many other moving parts compared to a standard software application, such as the data, model, and, finally, the code. MLOps aims to track, operationalize, and monitor all these concepts for better reproducibility, robustness, and control.

In ML systems, a build can be triggered by any change in these areas—whether it's an update in the code, modifications in the data, or adjustments to the model.

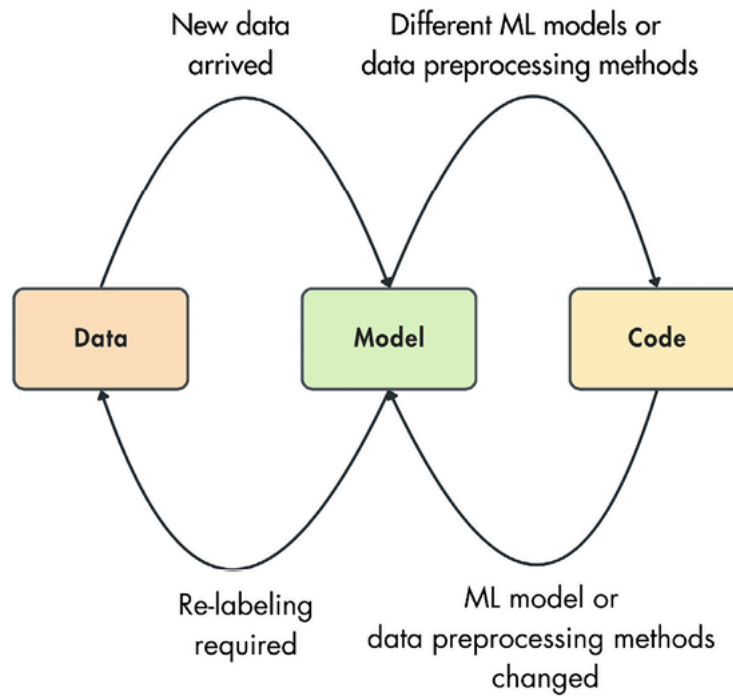


Figure 11.2: Relationship between data, model, and code changes

In DevOps, everything is centered around the code. For example, when a new feature is added to the codebase, you have to trigger the CI/CD pipeline. In MLOps, the code can remain unchanged while only the data changes. In that case, you must train (or fine-tune) a new model, resulting in a new dataset and model version. Intuitively, when one component changes, it affects one or more of the others. Thus, MLOps has to take into consideration all this extra complexity. Here are a few examples that can trigger a change in the data and indirectly in the model:

- After deploying the ML model, its performance might decay as time passes, so we need new data to retrain it.
- After understanding how to collect data in the real world, we might recognize that getting the data for our problem is challenging, so we need to re-formulate it to work with our real-world setup.
- While in the experimentation stage and training the model, we often must collect more data or re-label it, which generates a new set of models.
- After serving the model in the production environment and collecting feedback from the end users, we might recognize that the assumptions we made for training the model are wrong, so we must change our model.

So, what is MLOps?

A more official definition of MLOps is the following: MLOps is the extension of the DevOps field that makes data and models their first-class citizen while preserving the DevOps methodology.

Like DevOps, MLOps originates from the idea that isolating ML model development from its deployment process (ML operations) diminishes the system's overall quality, transparency, and agility. With that in mind, an optimal MLOps experience treats ML assets consistently as other software assets within a CI/CD environment as part of a cohesive release process.

MLOps core components

We have already used all of these components throughout the book, but let's have a quick refresher on the MLOps core components now that we better understand the field. Along with source control and CI/CD, MLOps revolves around:

- **Model registry:** A centralized repository for storing trained ML models (**tools: Comet ML, W&B, MLflow, ZenML**)
- **Feature store:** Preprocessing and storing input data as features for both model training and inference pipelines (**tools: Hopsworks, Tecton, Featureform**)
- **ML metadata store:** This store tracks information related to model training, such as model configurations, training data, testing data, and performance metrics. It is mainly used to compare multiple models and look at the model lineages to understand how they were created (**tools: Comet ML, W&B, MLflow**)
- **ML pipeline orchestrator:** Automating the sequence of steps in ML projects (**tools: ZenML, Airflow, Prefect, Dagster**)

You might have noticed an overlap between the MLOps components and its specific tooling. This is common, as most MLOps tools offer unified solutions, often called MLOps platforms.

MLOps principles

Six core principles guide the MLOps field. These are independent of any tool and sit at the core of building robust and scalable ML systems.

They are:

- **Automation or operationalization:** Automation in MLOps involves transitioning from manual processes to automated pipelines through CT and CI/CD. This enables the efficient retraining and deployment of ML models in response to triggers such as new data, performance drops, or unhandled edge cases. Moving from manual experimentation to full automation ensures that our ML systems are robust, scalable, and adaptable to changing requirements without errors or delays.
- **Versioning:** In MLOps, it is crucial to track changes in code, models, and data individually, ensuring consistency and reproducibility. Code is tracked using tools like Git, models are versioned through model registries, and data versioning can be managed using solutions like DVC or artifact management systems.
- **Experiment tracking:** As training ML models is an iterative and experimental process that involves comparing multiple experiments based on predefined metrics, using an experiment tracker to help us pick the best model is important. Tools like Comet ML, W&B, MLflow, and Neptune allow us to log all necessary information to compare experiments easily and select the best model for production.
- **Testing:** MLOps suggests that along with testing your code, you should also test your data and models through unit, integration, acceptance, regression, and stress tests. This ensures that each component functions correctly and integrates well, focusing on inputs, outputs, and handling edge cases.
- **Monitoring:** This stage is vital for detecting performance degradation in served ML models due to changes in production data, allowing

timely intervention such as retraining, further prompt or feature engineering, or data validation. By tracking logs, system metrics, and model metrics and detecting drifts, we can maintain the health of ML systems in production, detect issues as fast as possible, and ensure they continue to deliver accurate results.

- **Reproducibility:** This ensures that every process (such as training or feature engineering) within your ML systems produces identical results when given the same input by tracking all the moving variables, such as code versions, data versions, hyperparameters, or any other type of configurations. Due to the non-deterministic nature of ML training and inference, setting well-known seeds when generating pseudo-random numbers is essential to achieving consistent outcomes and making processes as deterministic as possible.

If you want to learn more, we've offered an in-depth exploration of these principles in the *Appendix* at the end of this book.

ML vs. MLOps engineering

There is a fine line between ML engineering and MLOps. If we want to define a rigid job description for the two roles, it cannot be easy to completely differentiate what responsibilities go into **ML engineering (MLE)** and what goes into MLOps. I have seen many job roles that bucket the MLOps role with the platform and cloud engineers. From one perspective, that makes a lot of sense: as an MLOps engineer, you have a lot of work to do on the infrastructure side. On the other hand, as seen in this section, an MLOps engineer still has to implement things such as experiment tracking, model registries, versioning, and more. A good strategy would be to let the ML engineer integrate these into the code and the MLOps engineer focus on making them work on their infrastructure.

At a big corporation, ultimately, differentiating the two roles might make sense. But when working in small to medium-sized teams, you will wear multiple hats and probably work on the ML system's MLE and MLOps aspects.

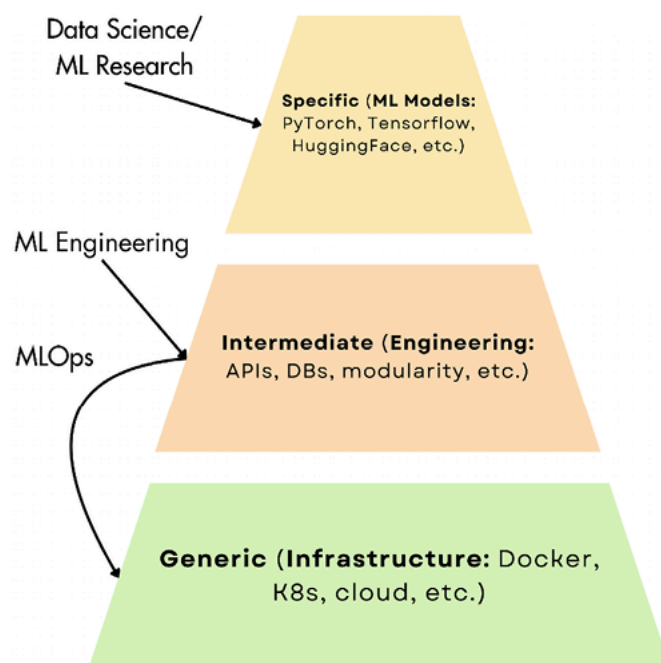


Figure 11.3: DS vs. MLE vs. MLOps

For instance, in *Figure 11.3*, we see a clear division of responsibilities among the three key roles: data scientist/ML researcher, ML engineer, and MLOps engineer. The **Data Scientist (DS)** implements specific models to address problems.

The ML engineer takes the functional models from the DS team and constructs a layer on top of them, making them modular and extendable and providing access to a **database (DB)** or exposing them as an API over the internet. However, the MLOps engineer plays a pivotal role in this process. They take the code from this intermediate layer and place it on a more generic layer, the infrastructure. This action marks the application's transition to production. From this point, we can start thinking about automation, monitoring, versioning, and more.

The intermediate layer differentiates a proof of concept from an actual product. In that layer, you design an extendable application that has a state by integrating a DB and is accessible over the internet through an API. When shipping the application on a specific infrastructure, you must consider scalability, latency, and cost-effectiveness. Of course, the intermediate and generic layers depend on each other, and often, you must re-iterate to meet the application requirements.

LLMOps

LLMOps encompasses the practices and processes essential for managing and running LLMs. This field is a specialized branch of MLOps, concentrating on the unique challenges and demands associated with LLMs. While MLOps addresses the principles and practices of managing various ML models, LLMOps focuses on the distinct aspects of LLMs, including their large size, highly complex training requirements, prompt management, and non-deterministic nature of generating answers. However, note that at its core, LLMOps still inherits all the fundamentals presented in the MLOps section. Thus, here, we will focus on what it adds on top.

When training LLMs from scratch, the data and model dimensions of an ML system grow substantially, which is one aspect that sets LLMOps apart from MLOps. These are the main concerns when training LLMs from scratch:

- **Data collection and preparation** involves collecting, preparing, and managing the massive datasets required for training LLMs. It involves big data techniques for processing, storing, and sharing training datasets. For example, GPT-4 was trained on roughly 13 trillion tokens, equal to approximately 10 trillion words.
- Managing **LLMs' considerable number of parameters** is a significant technical challenge from the infrastructure's point of view. It requires vast computation resources, usually clusters of machines powered by Nvidia GPUs with CUDA support.
- The massive size of LLMs directly impacts **model training**. When training an LLM from scratch, you can't fit it on a single GPU due to the model's size or the higher batch size you require for the expected results. Thus, you need multi-GPU training, which involves optimizing your processes and infrastructure to support data, model, or tensor parallelism.

- Managing massive datasets and multi-GPU clusters involves substantial **costs**. For example, the estimated training cost for GPT-4 is around \$100 million, as stated by Sam Altman, the CEO of OpenAI (<https://en.wikipedia.org/wiki/GPT-4#Training>). Add to that the costs of multiple experiments, evaluation, and inference. Even if these numbers are not exact, as the sources are not 100% reliable, the scale of the costs of training an LLM is trustworthy, which implies that only the large players in the industry can afford to train LLMs from scratch.

At its core, LLMOps is MLOps at scale. It uses the same MLOps principles but is applied to big data and huge models that require more computing power to train and run. However, due to its huge scale, the most significant trend is the shift away from training neural networks from scratch for specific tasks. This approach is becoming obsolete with the rise of fine-tuning, especially with the advent of foundation models such as GPT. A few organizations with extensive computational resources, such as OpenAI and Google, develop these foundation models. Thus, most applications now rely on the lightweight fine-tuning of parts of these models, prompt engineering, or optionally distilling data or models into smaller, specialized inference networks.

Thus, for most LLM applications out there, your development steps will involve the selection of a foundation model, which you further have to optimize by using prompt engineering, fine-tuning, or RAG. Thus, the operational aspect of these three steps is the most critical to understand. Let's dive into some popular components of LLMOps that can improve prompt engineering, fine-tuning, and RAG.

Human feedback

One valuable refinement step of your LLM is aligning it with your audience's preferences. You must introduce a feedback loop within your application and gather a human feedback dataset to further fine-tune the LLM with techniques such as **Reinforcement Learning with Human Feedback (RLHF)** or more advanced ones such as **Direct Preference Optimization (DPO)**. One popular feedback loop is the thumbs-up/thumbs-down button present in most chatbot interfaces. You can read more on preference alignment in *Chapter 6*.

Guardrails

Unfortunately, LLM systems are not reliable, as they often hallucinate. You can optimize your system against hallucinations, but as hallucinations are hard to detect and can take many forms, there are significant changes that will still happen in the future.

Most users have accepted this phenomenon, but what is not acceptable is when LLMs accidentally output sensitive information, such as GitHub Copilot outputting AWS secret keys or other chatbots providing people's passwords. This can also happen with people's phone numbers, addresses, email addresses, and more. Ideally, you should remove all this sensitive data from your training data so the LLM doesn't memorize it, but that doesn't always happen.

LLMs are well known for producing toxic and harmful outputs, such as sexist and racist outputs. For example, during an experiment on ChatGPT

around April 2023, people found how to hijack the system by forcing the chatbot to adopt a negative persona, such as “a bad person” or “a horrible person.” It worked even by forcing the chatbot to play the role of well-known negative characters from our history, such as dictators or criminals. For example, this is what ChatGPT produced when impersonating a bad person:

```
X is just another third-world country with nothing but drug lords and poverty-stric
```

Check the source of the experiment for more examples of different personas: <https://techcrunch.com/2023/04/12/researchers-discover-a-way-to-make-chatgpt-consistently-toxic/>.

The discussion can be extended to a never-ending list of examples, but the key takeaway is that your LLM can produce harmful output or receive dangerous input, so you should monitor and prepare for them. Thus, to create safe LLM systems, you must protect them against harmful, sensitive, or invalid input and output by adding guardrails:

- **Input guardrails:** Input guardrails primarily protect against three main risks: exposing private information to external APIs, executing harmful prompts that could compromise your system (model jailbreaking), and accepting violent or unethical prompts. When it comes to leaking private information to external APIs, the risk is specific to sending sensitive data outside your organization, such as credentials or classified information. When talking about model jailbreaking, we mainly refer to prompt injection, such as executing malicious SQL code that can access, delete, or corrupt your data. Lastly, some applications don't want to accept violent or unethical queries from users, such as asking an LLM how to build a bomb.
- **Output guardrails:** At the output of an LLM response, you want to catch failed outputs that don't respect your application's standards. This can vary from one application to another, but some examples are empty responses (these responses don't follow your expected format, such as JSON or YAML), toxic responses, hallucinations, and, in general, wrong responses. Also, you have to check for sensitive information that can leak from the internal knowledge of the LLM or your RAG system.

Popular guardrail tools are Galileo Protect, which detects prompt injections, toxic language, data privacy protection leaks, and hallucinations. Also, you can use OpenAI's Moderation API to detect harmful inputs or outputs and take action on them.

The downside of adding input and output guardrails is the extra latency added to your system, which might interfere with your application's user experience. Thus, there is a trade-off between the safety of your input/output and latency. Regarding invalid outputs, as LLMs are non-deterministic, you can implement a retry mechanism to generate another potential candidate. However, as stated above, running the retry sequentially will double the response time. Thus, a common strategy is to run multiple generations in parallel and pick the best one. This will increase redundancy but help keep the latency in check.

Prompt monitoring

Monitoring is not new to LLMOps, but in the LLM world, we have a new entity to manage: the prompt. Thus, we have to find specific ways to log and analyze them.

Most ML platforms, such as Opik (from Comet ML) and W&B, or other specialized tools like Langfuse, have implemented logging tools to debug and monitor prompts. While in production, using these tools, you usually want to track the user input, the prompt templates, the input variables, the generated response, the number of tokens, and the latency.

When generating an answer with an LLM, we don't wait for the whole answer to be generated; we stream the output token by token. This makes the entire process snappier and more responsive. Thus, when it comes to tracking the latency of generating an answer, the final user experience must look at this from multiple perspectives, such as:

- **Time to First Token (TTFT):** The time it takes for the first token to be generated
- **Time between Tokens (TBT):** The interval between each token generation
- **Tokens per Second (TPS):** The rate at which tokens are generated
- **Time per Output Token (TPOT):** The time it takes to generate each output token
- **Total Latency:** The total time required to complete a response

Also, tracking the total input and output tokens is critical to understanding the costs of hosting your LLMs.

Ultimately, you can compute metrics that validate your model's performance for each input, prompt, and output tuple. Depending on your use case, you can compute things such as accuracy, toxicity, and hallucination rate. When working with RAG systems, you can also compute metrics relative to the relevance and precision of the retrieved context.

Another essential thing to consider when monitoring prompts is to log their full traces. You might have multiple intermediate steps from the user query to the final general answer. For example, rewriting the query to improve the RAG's retrieval accuracy evolves one or more intermediate steps. Thus, logging the full trace reveals the entire process from when a user sends a query to when the final response is returned, including the actions the system takes, the documents retrieved, and the final prompt sent to the model. Additionally, you can log the latency, tokens, and costs at each step, providing a more fine-grained view of all the steps.

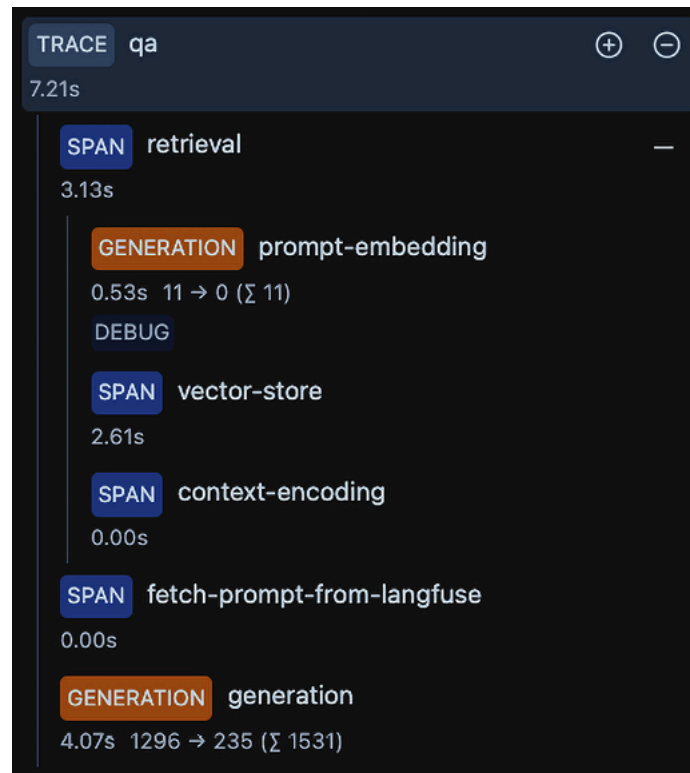


Figure 11.4: Example trace in the Langfuse UI

As shown in *Figure 11.4*, the end goal is to trace each step from the user's input until the generated answer. If something fails or behaves unexpectedly, you can point exactly to the faulty step. The query can fail due to an incorrect answer, an invalid context, or incorrect data processing. Also, the application can behave unexpectedly if the number of generated tokens suddenly fluctuates during specific steps.

To conclude, LLMops is a rapidly developing field. Given its quick evolution, making predictions is challenging. The truth is that we are not sure if the term LLMops is here to stay. However, what is certain is that numerous new use cases for LLMs will emerge, along with tools and best practices to manage their lifecycle.

Even if this DevOps, MLOps, and LLMops section is far from comprehensive, it provides a strong idea of how to apply best ops practices in our LLM Twin use case.

Deploying the LLM Twin's pipelines to the cloud

This section will show you how to deploy all the LLM Twin's pipelines to the cloud. We must deploy the entire infrastructure to have the whole system working in the cloud. Thus, we will have to:

1. Set up an instance of MongoDB serverless.
2. Set up an instance of Qdrant serverless.
3. Deploy the ZenML pipelines, container, and artifact registry to AWS.
4. Containerize the code and push the Docker image to a container registry.

Note that the training and inference pipelines already work with AWS SageMaker. Thus, by following the preceding four steps, we ensure that

our whole system is on the cloud, ready to scale and serve our imaginary clients.



What are the deployment costs?

We will stick to the free versions of the MongoDB, Qdrant, and ZenML services. As for AWS, we will mostly stick to their free tier for running the ZenML pipelines. The SageMaker training and inference components are more costly to run (which we won't run in this section). Thus, what we will show you in the following sections will generate minimum costs (a few dollars at most) from AWS.

Understanding the infrastructure

Before diving into the step-by-step tutorial, where we will show you how to set up all the necessary components, let's briefly overview our infrastructure and how all the elements interact. This will help us in mindfully following the tutorials below.

As shown in *Figure 11.5*, we have a few services to set up. To keep things simple, for MongoDB and Qdrant, we will leverage their serverless freemium version. As for ZenML, we will leverage the free trial of the ZenML cloud, which will help us orchestrate all the pipelines in the cloud. How will it do that?

By leveraging the ZenML cloud, we can quickly allocate all the required AWS resources to run, scale, and store the ML pipeline. It will help us spin up, with a few clicks, the following AWS components:

- An ECR service for storing Docker images
- An S3 object storage for storing all our artifacts and models
- SageMaker Orchestrator for orchestrating, running, and scaling all our ML pipelines

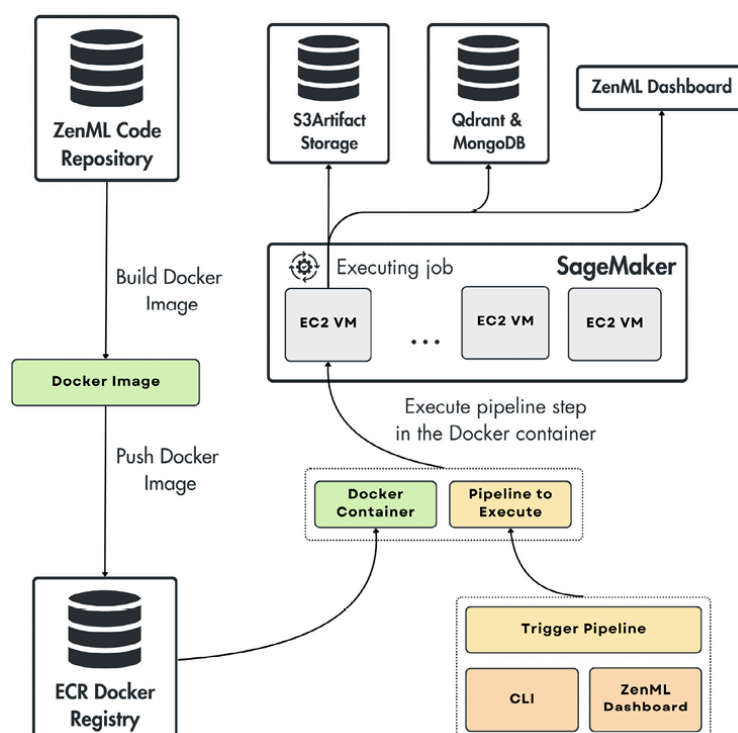


Figure 11.5: Infrastructure flow

Now that we understand what the essential resources of our infrastructure are, let's look over the core flow of running a pipeline in the cloud that we will learn to implement, presented in *Figure 11.5*:

1. Build a Docker image that contains all the system dependencies, the project dependencies, and the LLM Twin application.
2. Push the Docker image to **ECR**, where **SageMaker** can access it.
3. Now, we can trigger any pipeline implemented during this book either from the CLI of our local machine or **ZenML's** dashboard.
4. Each step from ZenML's pipeline will be mapped to a SageMaker job that runs on an AWS EC2 **virtual machine (VM)**. Based on the dependencies between the **directed acyclic graph (DAG)** steps, some will run in parallel and others sequentially.
5. When running a step, SageMaker pulls the Docker image from ECR, defined in step 2. Based on the pulled image, it creates a Docker container that executes the pipeline step.
6. As the job is executed, it can access the S3 artifact storage, MongoDB, and Qdrant vector DB to query or push data. The ZenML dashboard is a key tool, providing real-time updates on the pipeline's progress and ensuring a clear view of the process.

Now that we know how the infrastructure works, let's start by setting up MongoDB, Qdrant, and the ZenML cloud.

What AWS cloud region should I choose?

In our tutorials, all the services will be deployed to AWS within the **Frankfurt (eu-central-1)** region. You can select another region, but be consistent across all the services to ensure faster responses between components and reduce potential errors.



How should I manage changes in the services' UIs?

Unfortunately, MongoDB, Qdrant, or other services may change their UI or naming conventions. As we can't update this book each time that happens, please refer to their official documentation to check anything that differs from our tutorial. We apologize for this inconvenience, but unfortunately, it is not in our control.

Setting up MongoDB

We will show you how to create and integrate a free MongoDB cluster into our projects. To do so, these are the steps you have to follow:

1. Go to their site at <https://www.mongodb.com> and create an account.
2. In the left panel, go to **Deployment** | **Database** and click **Build a Cluster**.
3. Within the creation form, do the following:
 1. Choose an **M0 Free** cluster.
 2. Call your cluster **twin**.
 3. Choose **AWS** as your provider.

4. Choose **Frankfurt (eu-central-1)** as your region. You can choose another region, but be careful to choose the same region for all future AWS services.
5. Leave the rest of the attributes with their default values.
6. In the bottom right, click the **Create Deployment** green button.
4. To test that your newly created MongoDB cluster works fine, we must connect to it from our local machine. We used the MongoDB VS Code extension to do so, but you can use any other tool. Thus, from their **Choose a connection method** setup flow, choose **MongoDB for VS Code**. Then, follow the steps provided on their site.
5. To connect, you must paste the DB connection URL in the VS Code extension (or another tool of your liking), which contains your username, password, and cluster URL, similar to this one:

```
mongodb+srv://<username>:<password>@twin.vhxy1.mongodb.net
```

. Make sure to save this URL somewhere you can copy it from later.
6. If you don't know or want to change your password, go to **Security** → **Quickstart** in the left panel. There, you can edit your login credentials. Be sure to save them somewhere safe, as you won't be able to access them later.
7. After verifying that your connections work, go to **Security** → **Network Access** in the left panel and click **ADD IP ADDRESS**. Then click **ALLOW ACCESS FROM ANYWHERE** and hit Confirm. Out of simplicity, we allow any machine from any IP to access our MongoDB cluster. This ensures that our pipelines can query or write to the DB without any additional complex networking setup. It's not the safest option for production, but for our example, it's perfectly fine.
8. The final step is to return to your project and open your `.env` file. Now, either add or replace the `DATABASE_HOST` variable with your MongoDB connection string. It should look something like this:

```
DATABASE_HOST= mongodb+srv://<username>:<password>@twin.vhxy1.mongodb.net
```

.

That's it! Now, instead of reading and writing from your local MongoDB, you will do it from the cloud MongoDB cluster we just created. Let's repeat a similar process with Qdrant.

Setting up Qdrant

We have to repeat a similar process to what we did for MongoDB. Thus, to create a Qdrant cluster and hook it to our project, follow these steps:

1. Go to Qdrant at <https://cloud.qdrant.io/> and create an account.
2. In the left panel, go to **Clusters** and click **Create**.
3. Fill out the cluster creation form with the following:
 1. Choose the **Free** version of the cluster.
 2. Choose **GCP** as the cloud provider (while writing the book, it was the only one allowed for a free cluster).
 3. Choose **Frankfurt** as the region (or the same region as you chose for MongoDB).
 4. Name the cluster **twin**.
 5. Leave the rest of the attributes with their default values and click **Create**.
4. Access the cluster in the **Data Access Control** section in the left panel.
5. Click **Create** and choose your **twin** cluster to create a new access token. Copy the newly created token somewhere safe, as you won't be

able to access it anymore.

6. You can run their example from **Usage Examples** to test that your connection works fine.
7. Go back to the **Clusters** section of Qdrant and open your newly created **twin** cluster. You will have access to the cluster's **endpoint**, which you need to configure Qdrant in your code.

You can visualize your Qdrant collections and documents by clicking **Open Dashboard** and entering your **API Key** as your password. The Qdrant cluster dashboard will now be empty, but after running the pipelines, you will see all the collections, as shown here:

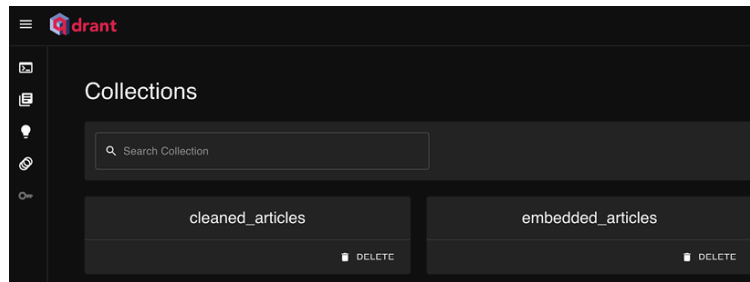


Figure 11.6: Qdrant cluster dashboard example after being populated with two collections.

Finally, return to your project and open your `.env` file. Now, we must fill in a couple of environment variables as follows:

```
USE_QDRANT_CLOUD=true
QDRANT_CLOUD_URL=<the endpoint URL found at step 7>
QDRANT_APIKEY=<the access token created at step 5>
```

That's it! Instead of reading and writing from your local Qdrant vector DB, you will do it from the cloud Qdrant cluster we just created. Just to be sure that everything works fine, run the end-to-end data pipeline with the cloud version of MongoDB and Qdrant as follows:

```
peotry poe run-end-to-end-data-pipeline
```

The last step is setting up the ZenML cloud and deploying all our infrastructure to AWS.

Setting up the ZenML cloud

Setting up the ZenML cloud and the AWS infrastructure is a multi-step process. First, we will set up a ZenML cloud account, then the AWS infrastructure through the ZenML cloud, and, finally, we will bundle our code in a Docker image to run it in AWS SageMaker.

Let's start with setting up the ZenML cloud:

1. Go to the ZenML cloud at <https://cloud.zenml.io> and make an account. They provide a seven-day free trial, which is enough to run our examples.
2. Fill out their onboarding form and create an organization with a unique name and a tenant called **twin**. A tenant refers to a deploy-

- ment of ZenML in a fully isolated environment. Wait a few minutes until your tenant server is up before proceeding to the next step.
3. If you want to, you can go through their **Quickstart Guide** to understand how the ZenML cloud works with a simpler example. It is not required to go through it to deploy the LLM Twin application, but we recommend it to ensure everything works fine.
 4. At this point, we assume that you have gone through the **Quickstart Guide**. Otherwise, you might encounter issues during the next steps. To connect our project with this ZenML cloud tenant, return to the project and run the `zenml connect` command provided in the dashboard. It looks similar to the following example but with a different URL: `zenml connect --url https://0c37a553-zenml.cloudinfra.zenml.io`.
 5. To ensure everything works fine, run a random pipeline from your code. Note that at this point, we are still running it locally, but instead of logging the results to the local server, we log everything to the cloud version:

```
poetry poe run-digital-data-etl
```

6. Go to the **Pipelines** section in the left panel of the ZenML dashboard. If everything worked fine, you should see the pipeline you ran in *Step 5* there.



Ensure that your ZenML server version matches your local ZenML version. For example, when we wrote this book, both were version 0.64.0. If they don't match, you might encounter strange behavior, or it might not work correctly. The easiest fix is to go to your `pyproject.toml` file, find the `zenml` dependency, and update it with the version of your server. Then run `poetry lock --no-update && poetry install` to update your local virtual environment.

To ship the code to AWS, you must create a ZenML stack. A stack is a set of components, such as the underlying orchestrator, object storage, and container registry, that ZenML needs under the hood to run the pipelines. Intuitively, you can see your stack as your infrastructure. While working locally, ZenML offers a default stack that allows you to quickly develop your code and test things locally. However, by defining different stacks, you can quickly switch between different infrastructure environments, such as local and AWS runs, which we will showcase in this section.



Before starting this section, ensure you have an AWS account with admin permissions ready.

With that in mind, let's create an AWS stack for our project. To do so, follow the next steps:

1. In the left panel, click on the **Stacks** section and hit the **New Stack** button.
2. You will have multiple options for creating a stack, but the easiest is creating one from scratch within the in-browser experience, which doesn't require additional preparations. This is not very flexible, but it

is enough to host our project. Thus, choose **Create New**

Infrastructure → **In-browser Experience**.

3. Then, choose **AWS** as your cloud provider.
4. Choose **Europe (Frankfurt)—eu-central-1** as your location or the region you used to set up MongoDB and Qdrant.
5. Name it **aws-stack**. It is essential to name it exactly like this so that the commands that we will use work.
6. Now ZenML will create a set of IAM roles to give permissions to all the other components to communicate with each other, an S3 bucket as your artifact storage, an ECR repository as your container registry, and SageMaker as your orchestrator.
7. Click **Next**.
8. Click the **Deploy to AWS** button. It will open a **CloudFormation** page on AWS. ZenML leverages **CloudFormation** (an infrastructure as code, or IaC, tool) to create all the AWS resources we enumerated in *Step 6*.
9. At the bottom, check all the boxes to acknowledge that AWS CloudFormation will create AWS resources on your behalf. Finally, click the **Create stack** button. Now, we must wait for a couple of minutes for AWS CloudFormation to spin up all the resources.
10. Return to the ZenML page and click the **Finish** button.

By leveraging ZenML, we efficiently deployed the entire AWS infrastructure for our ML pipelines. We began with a basic example, sacrificing some control. However, if you seek more control, ZenML offers the option to use Terraform (an IaC tool) to fully control your AWS resources or to connect ZenML with your current infrastructure.

Before moving to the next step, let's have a quick recap of the AWS resources we just created:

- **An IAM role** is an AWS identity with permissions policies that define what actions are allowed or denied for that role. It is used to grant access to AWS services without needing to share security credentials.
- **S3** is a scalable and secure object storage service that allows storing and retrieving files from anywhere on the web. It is commonly used for data backup, content storage, and data lakes. It's more scalable and flexible than Google Drive.
- **ECR** is a fully managed Docker container registry that makes storing, managing, and deploying Docker container images easy.
- **SageMaker** is a fully managed service that allows developers and data scientists to quickly build, train, and deploy ML models.
- **SageMaker Orchestrator** is a feature of SageMaker that helps automate the execution of ML workflows, manage dependencies between steps, and ensure the reproducibility and scalability of model training and deployment pipelines. Other similar tools are Prefect, Dagster, Metaflow, and Airflow.
- **CloudFormation** is a service that allows you to model and set up your AWS resources so that you can spend less time managing them and more time focusing on your applications. It automates the process of provisioning AWS infrastructure using templates.

Before running the ML pipelines, the last step is to containerize the code and prepare a Docker image that packages our dependencies and code.

Containerize the code using Docker

So far, we have defined our infrastructure, MongoDB, Qdrant, and AWS, for storage and computing. The last step is to find a way to take our code and run it on top of this infrastructure. The most popular solution is Docker, a tool that allows us to create an isolated environment (a container) that contains everything we need to run our application, such as system dependencies, Python dependencies, and the code.

We defined our Docker image at the project's root in the `Dockerfile`. This is the standard naming convention for Docker. Before digging into the code, if you want to build the Docker image yourself, ensure that you have Docker installed on your machine. If you don't have it, you can install it by following the instructions provided here: <https://docs.docker.com/engine/install>. Now, let's look at the content of the `Dockerfile` step by step.

The `Dockerfile` begins by specifying the base image, which is a lightweight version of Python 3.11 based on the Debian Bullseye distribution. The environment variables are then set up to configure various aspects of the container, such as the workspace directory, turning off Python bytecode generation, and configuring Python to output directly to the terminal. Additionally, the version of Poetry to be installed is specified, and a few environment variables are set to ensure that package installations are non-interactive, which is vital for automated builds.

```
FROM python:3.11-slim-bullseye AS release
ENV WORKSPACE_ROOT=/app/
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
ENV POETRY_VERSION=1.8.3
ENV DEBIAN_FRONTEND=noninteractive
ENV POETRY_NO_INTERACTION=1
```

Next, we install Google Chrome in the container. The installation process begins by updating the package lists and installing essential tools like `gnupg`, `wget`, and `curl`. The Google Linux signing key is added, and the Google Chrome repository is configured. After another package list update, the stable version of Google Chrome is installed. The package lists are removed after installation to keep the image as small as possible.

```
RUN apt-get update -y && \
    apt-get install -y gnupg wget curl --no-install-recommends && \
    wget -q -O - https://dl-ssl.google.com/linux/linux_signing_key.pub | gpg --dearmor \
    echo "deb [signed-by=/usr/share/keyrings/google-linux-signing-key.gpg] https://dl.google.com/linux/chrome/deb/ stable main" > /etc/apt/sources.list.d/google-chrome.list \
    apt-get update -y && \
    apt-get install -y google-chrome-stable && \
    rm -rf /var/lib/apt/lists/*
```

 **Quick tip:** Enhance your coding experience with the **AI Code Explainer** and **Quick Copy** features. Open this book in the next-gen Packt Reader. Click the **Copy** button (1) to quickly copy code into your coding environment, or click the **Explain** button (2) to get the AI assistant to explain a block of code to you.



 **The next-gen Packt Reader** is included for free with the purchase of this book. Unlock it by scanning the QR code below.

low or visiting

<https://www.packtpub.com/unlock/9781836200079>.

Following the Chrome installation, other essential system dependencies are installed. Once these packages are installed, the package cache is cleaned up to reduce the image size further.

```
RUN apt-get update -y \
    && apt-get install -y --no-install-recommends build-essential \
    gcc \
    python3-dev \
    build-essential \
    libglib2.0-dev \
    libnss3-dev \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*
```

Poetry, the dependency management tool, is then installed using pip. The `--no-cache-dir` option prevents pip from caching packages, helping to keep the image smaller. After installation, Poetry is configured to use up to 20 parallel workers when installing packages, which can speed up the installation process.

```
RUN pip install --no-cache-dir "poetry==$POETRY_VERSION"
RUN poetry config installer.max-workers 20
```

The working directory inside the container is set to `WORKSPACE_ROOT`, which defaults to `/app/`, where the application code will reside. The `pyproject.toml` and `poetry.lock` files define the Python's project dependencies and are copied into this directory.

```
WORKDIR $WORKSPACE_ROOT
COPY pyproject.toml poetry.lock $WORKSPACE_ROOT
```

With the dependency files in place, the project's dependencies are installed using Poetry. The configuration turns off the creation of a virtual environment, meaning the dependencies will be installed directly into the container's Python environment. The installation excludes development dependencies and prevents caching to minimize space usage. Additionally, the `poethepoet` plugin is installed to help manage tasks within the project. Finally, any remaining Poetry cache is removed to keep the container as lean as possible.

```
RUN poetry config virtualenvs.create false && \
    poetry install --no-root --no-interaction --no-cache --without dev && \
    poetry self add 'poethepoet[poetry_plugin]' && \
    rm -rf ~/.cache/pypoetry/cache/ && \
    rm -rf ~/.cache/pypoetry/artifacts/
```

In the final step, the entire project directory from the host machine is copied into the container's working directory. This step ensures that all the application files are available within the container.

One important trick when writing a `Dockerfile` is to decouple your installation steps from copying the rest of the files. This is useful because each Docker command is cached and layered on top of each other. Thus, whenever you change one layer when rebuilding the Docker image, all the layers below the one altered are executed again. Because you rarely change your system and project dependencies but mostly change your code, copying your project files in the last step makes rebuilding Docker images fast by taking advantage of the caching mechanism's full potential.

```
COPY . $WORKSPACE_ROOT
```

This `Dockerfile` is designed to create a clean, consistent Python environment with all necessary dependencies. It allows the project to run smoothly in any environment that supports Docker.

The last step is to build the Docker image and push it to the ECR created by ZenML. To build the Docker image from the root of the project, run the following:

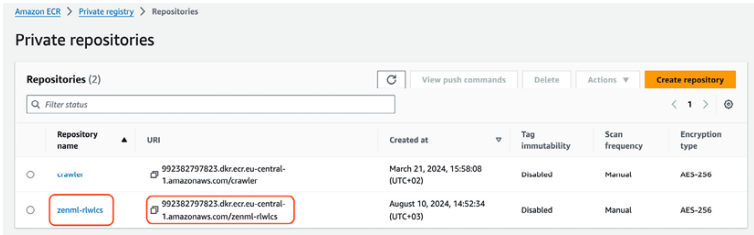
```
docker buildx build --platform linux/amd64 -t llmtwin -f Dockerfile .
```

We must build it on a Linux platform as the Google Chrome installer we used inside Docker works only on a Linux machine. Even if you use a macOS or Windows machine, Docker can emulate a virtual Linux container.

The tag of the newly created Docker image is `llmtwin`. We also provide this `build` command under a `poethepoet` command:

```
poetry poe build-docker-image
```

Now, let's push the Docker image to ECR. To do so, navigate to your AWS console and then to the ECR service. From there, find the newly created ECR repository. It should be prefixed with `zenml-*`, as shown here:



Repository name	URI	Created at	Tag immutability	Scan frequency	Encryption type
crawler	992382797823.dkr.ecr.eu-central-1.amazonaws.com/crawler	March 21, 2024, 15:58:08 (UTC+02)	Disabled	Manual	AES-256
zenml-rlwics	992382797823.dkr.ecr.eu-central-1.amazonaws.com/zenml-rlwics	August 10, 2024, 14:52:34 (UTC+03)	Disabled	Manual	AES-256

Figure 11.7: AWS ECR example

The first step is to authenticate to ECR. For this to work, ensure that you have the AWS CLI installed and configured with your admin AWS credentials, as explained in *Chapter 2*:

```
AWS_REGION=<your_region> # e.g. AWS_REGION=eu-central-1
AWS_ECR_URL=<your_account_id>
aws ecr get-login-password --region ${AWS_REGION} | docker login --username AWS --password-
```

You can get your current `AWS_REGION` by clicking on the toggle in the top-right corner, as seen in *Figure 11.8*. Also, you can copy the ECR URL to fill the `AWS_ECR_URL` variable from the main AWS ECR dashboard, as illustrated in *Figure 11.7*. After running the previous command, you should see the message **Login Succeeded** on the CLI.

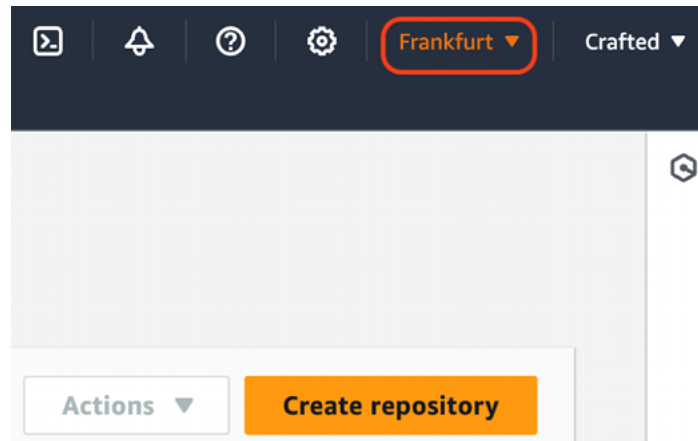


Figure 11.8: AWS region and account details

Now we have to add another tag to the `llmtwin` Docker image that signals the Docker registry we want to push it to:

```
docker tag llmtwin ${AWS_ECR_URL}:latest
```

Finally, we push it to ECR by running:

```
docker push ${AWS_ECR_URL}:latest
```

After the upload is finished, return to your AWS ECR dashboard and open your ZenML repository. The Docker image should appear, as shown here:

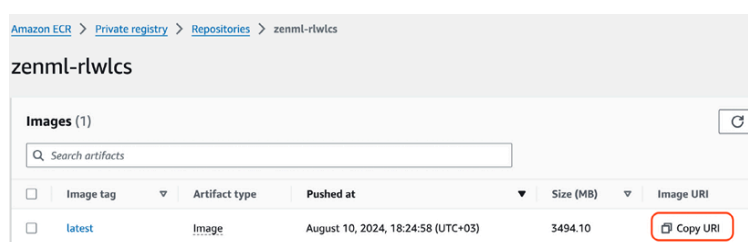


Figure 11.9: AWS ECR repository example after the Docker image is pushed

For every change in the code that you need to ship and test, you would have to go through all these steps, which are tedious and error-prone. The *Adding LLMops to the LLM Twin* section of this chapter will teach us how to automate these steps within the CD pipeline using GitHub Actions.

Still, we first wanted to go through them manually to fully understand the behind-the-scenes process and not treat it as a black box. Understanding these details is vital for debugging your CI/CD pipelines, where you must understand the error messages and how to fix them.

Now that we have built our Docker image and pushed it to AWS ECR, let's deploy it to AWS.

Run the pipelines on AWS

We are very close to running the ML pipelines on AWS, but we have to go through a few final steps. Let's switch from the default ZenML stack to the AWS one we created in this chapter. From the root of your project, run the following in the CLI:

```
zenml stack set aws-stack
```

Return to your AWS ECR ZenML repository and copy the image URI as shown in *Figure 11.9*. Then, go to the `configs` directory, open the `configs/end_to_end_data.yaml` file, and update the `settings.docker.parent_image` attribute with your ECR URL, as shown below:

```
settings:
  docker:
    parent_image: <YOUR ECR URL> #e.g., 992382797823.dkr.ecr.eu-central-1.amazonaws.com
    skip_build: True
```

We've configured the pipeline to always use the latest Docker image available in ECR. This means that the pipeline will automatically pick up the latest changes made to the code whenever we push a new image.

We must export all the credentials from our `.env` file to ZenML secrets, a feature that safely stores your credentials and makes them accessible within your pipelines:

```
poetry poe export-settings-to-zenml
```

The last step is setting up to run the pipelines asynchronously so we don't have to wait until they are finished, which might result in timeout errors:

```
zenml orchestrator update aws-stack --synchronous=False
```

Now that ZenML knows to use the AWS stack, our custom Docker image, and has access to our credentials, we are finally done with the setup. Run the `end-to-end-data-pipeline` with the following command:

```
poetry poe run-end-to-end-data-pipeline
```

Now you can go to **ZenML Cloud** → **Pipelines** → **end_to_end_data** and open the latest run. On the ZenML dashboard, you can visualize the latest state of the pipeline, as seen in *Figure 11.10*. Note that this pipeline runs all the data-related pipelines in a single run.

In the *Adding LLMops to the LLM Twin* section, we will explain why we compressed all the steps into a single pipeline.

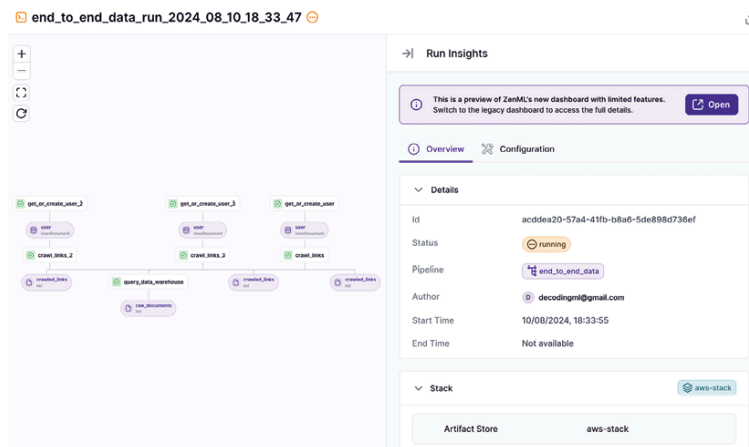


Figure 11.10: ZenML example of running the end-to-end-data-pipeline

Quick tip: Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

The next-gen Packt Reader and a **free PDF/ePub copy** of this book are included with your purchase. Unlock them by scanning the QR code below or visiting <https://www.packtpub.com/unlock/9781836200079>.

You can click on any running block and find details about the run, the code used for that specific step, and the logs for monitoring and debugging, as illustrated in Figure 11.11:

query_data_warehouse completed

Overview Code Logs Configuration

Details

Orchestrator url	Not available
Orchestrator run id	arn:aws:sagemaker:eu-central-1:992382797823:pipeline/end-to-end-data-b
Id	6876bd76-e1f6-4e62-bc8c-c8f5d0f87659
Status	Completed
Pipeline	end_to_end_data
Code Path	s3://zenml-rlwlc-992382797823/code_uploads/8524b9c5e855a0a41cf4c34a90120724268698fb

Figure 11.11: ZenML step metadata example

To run other pipelines, you have to update the `settings.docker.parent_image` attribute in their config file under the `configs/` directory.

To find even more details about the runs, you can go to AWS SageMaker. In the left panel, click **SageMaker dashboard**, and on the right, in the **Processing** column, click on the green **Running** section, as shown in Figure 11.12.

This will open a list of all the **processing jobs** that execute your ZenML pipelines.

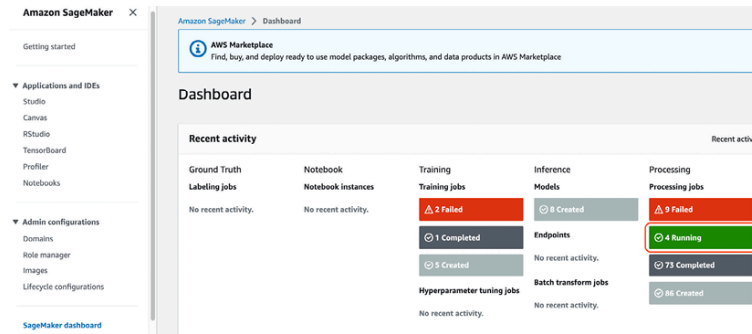


Figure 11.12: SageMaker dashboard



If you want to run the pipelines locally again, use the following CLI command:

```
poetry poe set-local-stack
```

If you want to disconnect from the ZenML cloud dashboard and use the local version again, run the following:

```
zenml disconnect
```

Troubleshooting the ResourceLimitExceeded error after running a ZenML pipeline on SageMaker

Let's assume, you've encountered a **ResourceLimitExceeded** error after running a ZenML pipeline on SageMaker using the AWS stack. In this case, you have to explicitly ask AWS to give you access to a specific type of AWS EC2 VM.

ZenML uses, by default, `m1.t3.medium` EC2 machines, which are part of the AWS freemium tier. However, some AWS accounts cannot access these VMs by default. To check your access, search your AWS console for **Service Quotas**. Then, in the left panel, click on **AWS services**, search for **Amazon SageMaker**, and then for `m1.t3.medium`. In *Figure 11.13*, you can see our quotas for these types of machines. If yours is **0**, you should request that AWS increase them to numbers similar to those from *Figure 11.13* in the **Applied account-level quota value** column. The whole process is free of charge and only requires a few clicks. Unfortunately, you might have to wait for a few hours up to one day until AWS accepts your request.

Service Quotas > AWS services > Amazon SageMaker

Amazon SageMaker

Amazon SageMaker is a fully-managed service that enables data scientists and developers to quickly and easily build, train, and deploy machine learning models at scale.

Service quotas info

View your applied quota values, default quota values, and request quota increases for quotas. [Learn more](#)

Search: ml.t3.medium 7 matches

Quota name	Applied account-level quota value	AWS default quota value	Utilization	Adjustability
ml.t3.medium for cluster usage	10	0	0	Account level
ml.t3.medium for notebook instance usage	30	6	0	Account level
ml.t3.medium for processing job usage	10	4	1	Account level
RSessionGateway Apps running on ml.t3.medium instance	30	2	0	Account level
Studio CodeEditor Apps running on ml.t3.medium instances	6	6	0	Account level
Studio JupyterLab Apps running on ml.t3.medium instances	6	6	0	Account level
Studio KernelGateway Apps running on ml.t3.medium instance	30	2	0	Account level

Figure 11.13: SageMaker—ml.t3.medium expected quotas

You can find step-by-step instructions on how to solve this error and request new quotas at this link: <https://repost.aws/knowledge-center/sagemaker-resource-limit-exceeded-error>.



If you changed the values from your .env file and want to update the ZenML secrets with them, first run the following CLI command to delete the old secrets:

```
poetry poe delete-settings-zenml
```

Then, you can export them again by running:

```
poetry poe export-settings-to-zenml
```

Adding LLMOps to the LLM Twin

In the previous section, we saw how to set up the infrastructure for the LLM Twin project by manually building the Docker image and pushing it to ECR. We want to automate the entire process and implement a CI/CD pipeline using GitHub Actions and a CT pipeline using ZenML. As mentioned earlier, implementing a CI/CD/CT pipeline ensures that each feature pushed to main branches is consistent and tested. Also, by automating the deployment and training, you support collaboration, save time, and reduce human errors.

Finally, at the end of the section, we will show you how to implement a prompt monitoring pipeline using Opik from Comet ML and an alerting system using ZenML. This prompt monitoring pipeline will help us debug and analyze the RAG and LLM logic. As LLM systems are non-deterministic, capturing and storing the prompt traces is essential for monitoring your ML logic.

Before diving into the implementation, let's start with a quick section on the LLM Twin's CI/CD pipeline flow.

LLM Twin's CI/CD pipeline flow

We have two environments: staging and production. When developing a new feature, we create a new branch out of the staging branch and develop solely on that one. When we are done and consider the feature finished, we open a **pull request (PR)** to the staging branch. After the fea-

ture branch is accepted, it is merged into the staging branch. This is a standard workflow in most software applications. There might be variations, like adding a dev environment, but the principles remain the same.

As illustrated in *Figure 11.14*, the CI pipeline is triggered when the PR opens. At this point, we test the feature branch for linting and formatting errors. Also, we run a `gitleaks` command to check for credentials and sensitive information that was committed by mistake. If the linting, formatting, and gitleaks steps pass (also known as static analysis), we run the automated tests. Note that the static analysis steps run faster than the automated tests. Thus, the order matters. That's why adding the static analysis steps at the beginning of the CI pipeline is good practice. We propose the following order of the CI steps:

- `gitleaks` checks
- Linting checks
- Formatting checks
- Automated testing, such as unit and integration tests

If any check fails, the CI pipeline fails, and the developer who created the PR cannot merge it into the staging branch until it fixes the issues.

Implementing a CI pipeline ensures that new features follow the repository's standards and don't break existing functionality. The exact process repeats when we plan to merge the staging branch into the production one. We open a PR, and the CI pipeline is automatically executed before merging the staging branch into production.

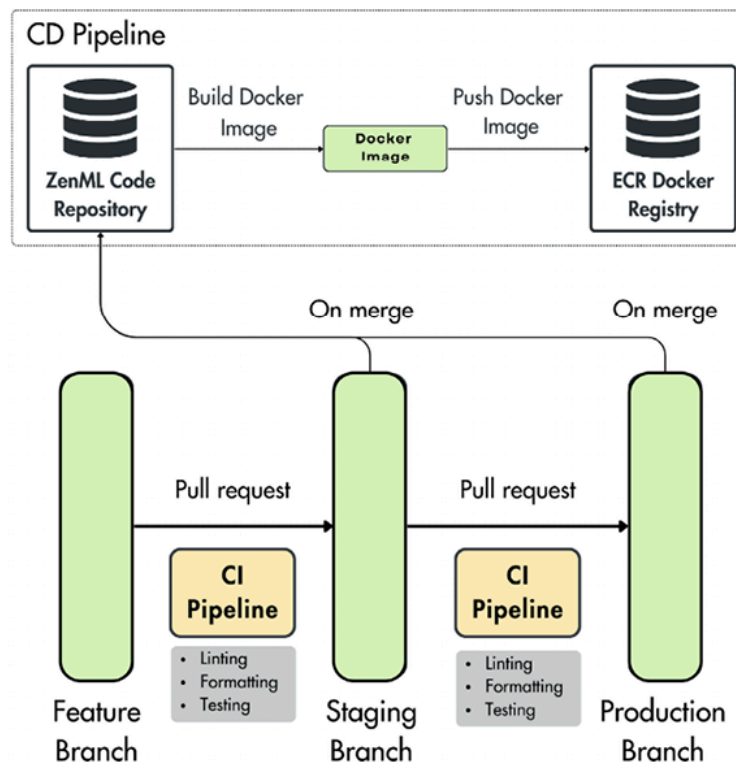


Figure 11.14: CI/CD pipelines flow

The CD pipeline runs after the branch is merged. For example, after the feature branch is merged into staging, the CD pipeline takes the code from the staging branch, builds a new Docker image, and pushes it to the AWS ECR Docker repository. When running future pipeline runs in the

staging environment, it will use the latest Docker image that was built by the CD pipeline. The exact process happens between staging and production. Still, the key difference is that the staging environment exists as an experimental place where the QA team and stakeholders can further manually test the new feature along with what is automatically tested in the CI pipeline.



In our repository, we used only a main branch, which reflects production, and feature branches to push new work. We did this to keep things simple, but the same principles apply. To extend the flow, you must create a staging branch and add it to the CD pipeline.

More on formatting errors

Formatting errors relate to the style and structure of your code, ensuring that it adheres to a consistent visual layout. This can include the placement of spaces, indentation, line length, and other stylistic elements.

The main purpose of formatting is to make your code more readable and maintainable. Consistent formatting helps teams work together more effectively, as the code looks uniform, regardless of who wrote it. Examples of formatting errors are:

- Incorrect indentation (e.g., mixing spaces and tabs)
- Lines that are too long (e.g., exceeding 79 or 88 characters, depending on your style guide)
- Missing or extra spaces around operators or after commas

More on linting errors

Linting errors relate to potential issues in your code that could lead to bugs, inefficiencies, or non-adherence to coding standards beyond just style. Linting checks often involve static analysis of the code to catch things like unused variables, undefined names, or questionable practices.

Linting's main goal is to catch potential errors or bad practices early in the development process, improving code quality and reducing the likelihood of bugs. Examples of linting errors are:

- Unused imports or variables
- Undefined variables or functions are being used
- Potentially dangerous code (e.g., using `==` instead of `is` for checking against `None`)

We use Ruff, a versatile tool for formatting and linting. It incorporates checks for common formatting issues and PEP 8 compliance, as well as deeper linting checks for potential errors and code quality problems. Also, it is written in Rust, making it fast for big codebases.

Before implementing what we've explained above, let's examine the core principles of GitHub Actions.

Quick overview of GitHub Actions

GitHub Actions is a CI/CD platform provided by GitHub that allows developers to automate their workflows directly within a GitHub repository. It

enables users to build, test, and deploy their code directly from GitHub by defining workflows in YAML files. Since it's part of GitHub, it works seamlessly with repositories, issues, PRs, and other GitHub features. Here are the key components you should know about:

- **Workflows:** A workflow is an automated process defined in a YAML file located in your repository's `.github/workflows` directory. It specifies what should happen (e.g., `build`, `test`, and `deploy`) and when (e.g., on push, on PR).
- **Jobs:** Workflows are made up of jobs, which are groups of steps that execute on the same runner. Each job runs in its own virtual environment.
- **Steps:** Jobs are made up of multiple independent steps, which can be actions or shell commands.
- **Actions:** Actions are reusable commands or scripts. You can use pre-built actions from GitHub Marketplace or create your own. You can think of them as Python functions.
- **Runners:** Runners are the servers that run your jobs. GitHub provides hosted runners (Linux, Windows, macOS), or you can even self-host your runners.

A workflow is described using YAML syntax. For example, a simple workflow that clones the current GitHub repository and installs Python 3.11 on an Ubuntu machine looks like this:

```
name: Example
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v3
      - name: Setup Python
        uses: actions/setup-python@v3
        with:
          python-version: "3.11"
```

The workflows are triggered by events like `push`, `pull_request`, or `schedule`. For example, you might trigger a workflow every time code is pushed to a specific branch. Now that we understand how GitHub Actions works, let's look at the LLM Twin's CI pipeline.

The CI pipeline

The LLM Twin's CI pipeline is split into two jobs:

- A **QA job** that looks for formatting and linting errors using Ruff. Also, it runs a `gitleaks` step to scan for leaked secrets throughout our repository.
- A **test job** that runs all our automatic tests using `Pytest`. In our use case, we implemented just a dummy test to showcase the CI pipeline, but using the structure from this book, you can easily extend it with real tests for your use case.

GitHub Actions CI YAML file

The YAML file sits under `.github/workflows/ci.yaml`. It begins by defining the workflow's name as `CI`, as you can see in the following snippet. This label will be used to identify the workflow within GitHub's Actions interface. Next, the section specifies that the workflow should be triggered whenever a `pull_request` event occurs. Hence, the CI workflow will automatically run whenever a PR is opened, synchronized, or reopened.

```
name: CI
on:
  pull_request:
```

The `concurrency` section ensures that only one instance of this workflow runs for a given reference (like a branch) at any given time. The `group` field is defined using GitHub's expression syntax to create a unique group name based on the workflow and the reference. The `cancel-in-progress: true` line ensures that if a new workflow run is triggered before the previous one finishes, the previous run is canceled. This is particularly useful to prevent redundant executions of the same workflow.

```
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

The workflow defines two separate jobs: `qa` and `test`. Each job runs on the latest version of Ubuntu, specified by `runs-on: ubuntu-latest`.

The first job, named `QA`, is responsible for quality assurance tasks like code checks and formatting verification. Within the `qa` job, the first step is to check out the repository's code using the `actions/checkout@v3` action. This step is necessary to ensure that the job has access to the code that needs to be analyzed.

```
jobs:
  qa:
    name: QA
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v3
```

The next step is to set up the Python environment. This is done using the `actions/setup-python@v3` action, with the Python version specified as `"3.11"`. This step ensures that the subsequent steps in the job will run in the correct Python environment.

```
- name: Setup Python
  uses: actions/setup-python@v3
  with:
    python-version: "3.11"
```

The workflow then installs Poetry using the `abato/actions-poetry@v2` action, specifying the version of Poetry as `1.8.3`:

```
- name: Install poetry
  uses: abatilo/actions-poetry@v2
  with:
    poetry-version: 1.8.3
```

Once Poetry is set up, the workflow installs the project's development dependencies using the `poetry install --only dev` command. Additionally, the workflow adds the `poethepoet` plugin for Poetry, which will be used to run predefined tasks more conveniently within the project.

```
- name: Install packages
  run: |
    poetry install --only dev
    poetry self add 'poethepoet[poetry_plugin]'
```

The `qa` job then runs several quality checks on the code. The first check uses a tool called `gitleaks` to scan for secrets in the codebase, ensuring that no sensitive information is accidentally committed:

```
- name: gitleaks check
  run: poetry poe gitleaks-check
```

Following the `gitleaks` check, the workflow runs a linting process to enforce coding standards and best practices in the Python code. This is achieved through the `poetry poe lint-check` command, which uses Ruff under the hood.

```
- name: Lint check [Python]
  run: poetry poe lint-check
```

The last step in the `qa` job is a format check, which ensures that the Python code is properly formatted according to the project's style guidelines. This is done using the `poetry poe format-check` command, which uses Ruff under the hood.

```
- name: Format check [Python]
  run: poetry poe format-check
```

The **second job** defined in the workflow is the `test` job, which also runs on the latest version of Ubuntu. Like the `qa` job, it starts by checking out the code from the repository and installing Python 3.11 and Poetry 1.8.3.

```
test:
  name: Test
  runs-on: ubuntu-latest
  steps:
    - name: Checkout
      uses: actions/checkout@v3
    ...
```

After setting up the system dependencies, the `test` job installs all the project's dependencies with the `poetry install` command. As we want

to run the tests, this time, we need to install all the dependencies that are required to run the application.

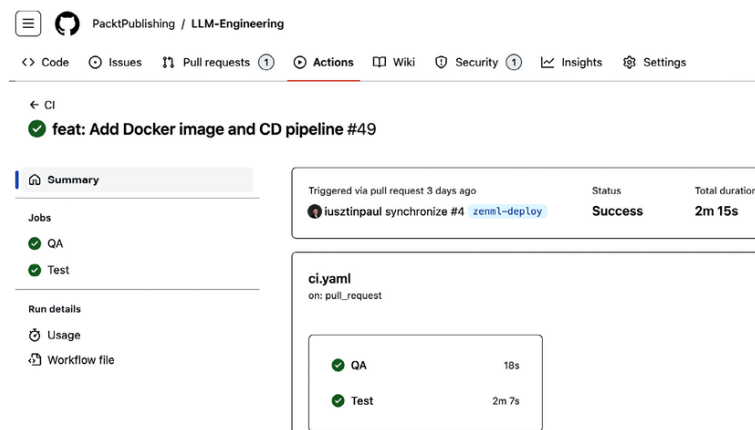
```
- name: Install packages
  run: |
    poetry install --without aws
    poetry self add 'poethepoet[poetry_plugin]'
```

Finally, the `test` job runs the project's tests using the `poetry poe test` command. This step ensures that all tests are executed and provides feedback on whether the current code changes break any functionality.

```
- name: Run tests
  run: |
    echo "Running tests..."
    poetry poe test
```

If any of the steps from the QA or test jobs fail, the GitHub Actions workflow will fail, resulting in the PR not being able to be merged until the issue is fixed. By taking this approach, we ensure that all the new features added to the main branches respect the standard of the project and that it doesn't break existing functionality through automated tests.

Figure 11.15 shows the CI pipeline in the **Actions** tab of the GitHub repository. It was run after a commit with the message **feat: Add Docker image and CD pipeline** and ran the two jobs described above, QA and Test.



The screenshot shows the GitHub Actions interface for a repository named 'PacktPublishing / LLM-Engineering'. The 'Actions' tab is selected, showing a workflow named 'feat: Add Docker image and CD pipeline #49' which has a green checkmark indicating success. The workflow was triggered via a pull request 3 days ago by user 'iusztinpaul' and completed successfully in 2m 15s. A summary section on the left lists the jobs: QA and Test, both with green checkmarks. On the right, a 'ci.yaml' file is shown with the trigger 'on: pull_request'. Below this, a table summarizes the job durations: QA took 18s and Test took 2m 7s.

Triggered via	Status	Total duration
pull request 3 days ago	Success	2m 15s

Job	Duration
QA	18s
Test	2m 7s

Figure 11.15: GitHub Actions CI pipeline run example

The CD pipeline

The CD pipeline will automate the Docker steps we manually performed in the **Deploying the LLM Twin's pipelines to the cloud** section, which are:

- Set up Docker.
- Log in to AWS.
- Build the Docker image.
- Push the Docker image to AWS ECR.

With that in mind, let's look at the GitHub Actions YAML file, which sits under `.github/workflows/cd.yaml`. It begins by naming the workflow `CD` and specifying the trigger for this workflow. The trigger is any push to the repository's main branch. This workflow will automatically run when new code is pushed to the main branch, usually when a PR is merged into the main branch. The `on.push` configuration sets up the trigger:

```
name: CD
on:
  push:
    branches:
      - main
```

The workflow then defines a single job named `Build & Push Docker Image`:

```
jobs:
  build:
    name: Build & Push Docker Image
    runs-on: ubuntu-latest
```

The first step within the job is to check out the repository's code.

```
steps:
  - name: Checkout Code
    uses: actions/checkout@v3
```

After checking out the code, the workflow sets up `docker buildx`, a Docker CLI plugin that extends Docker's build capabilities with features like multi-platform builds and cache import/export:

```
- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3
```

The next step involves configuring the AWS credentials. This step is crucial for interacting with AWS services, such as Amazon **Elastic Container Registry (ECR)**, where the Docker images will be pushed. The AWS access key, secret access key, and region are securely retrieved from the repository's secrets to authenticate the workflow with AWS. This ensures the workflow has the necessary permissions to push Docker images to the ECR repository. We will show you how to configure these secrets after wrapping up with the YAML file:

```
- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v1
  with:
    aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
    aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
    aws-region: ${ secrets.AWS_REGION }
```

Once the AWS credentials are configured, the workflow logs in to Amazon ECR. This step is essential for authenticating the Docker CLI with the ECR registry, allowing subsequent steps to push images to the registry:

```

- name: Login to Amazon ECR
  id: login-ecr
  uses: aws-actions/amazon-ecr-login@v1

```

The final step in the workflow involves building the Docker image and pushing it to the Amazon ECR repository. This is accomplished using the `docker/build-push-action@v6` action. The `context` specifies the build context, which is typically the repository's root directory. The `file` option points to the `Dockerfile`, which defines how the image should be built. The `tags` section assigns tags to the image, including the specific commit SHA and the `latest` tag, which is a common practice for identifying the most recent version of the image. The `push` option is set to `true`, meaning the image will be uploaded to ECR after it is built:

```

- name: Build images & push to ECR
  id: build-image
  uses: docker/build-push-action@v6
  with:
    context: .
    file: ./Dockerfile
    tags: |
      ${ steps.login-ecr.outputs.registry }/${ secrets.AWS_ECR_NAME }:${ gith
      ${ steps.login-ecr.outputs.registry }/${ secrets.AWS_ECR_NAME }:latest
    push: true

```

To conclude, the CD pipeline authenticates to AWS, builds the Docker image, and pushes it to AWS ECR. The Docker image is pushed with `latest` and the commit's SHA tag. By doing so, we can always use the latest image and point to the commit of the code from which the image was generated.

Also, in our code, we have only a main branch, which reflects our production environment. But you, as a developer, have the power to extend this functionality with a staging and dev environment. You just have to add the name of the branches in the `on.push.branches` configuration at the beginning of the YAML file.

In *Figure 11.16*, you can observe how the CD pipeline looks after a PR is merged into the production branch. As seen before, we only have the **Build & Push Docker Image** job here.

The screenshot shows the GitHub Actions interface for a workflow named 'CD' in the repository 'PacktPublishing / LLM-Engineering'. The workflow is triggered by a pull request merge. The summary shows a successful run of the 'Build & Push Docker Image' job, triggered 21 minutes ago by user 'iusztinpaul' on the 'main' branch. The job status is 'Success', with a total duration of 7m 10s and 1 artifact. The workflow file 'cd.yaml' is shown, with the trigger set to 'on: push'. A job card for 'Build & Push Docker Im...' is visible at the bottom.

Figure 11.16: GitHub Actions CD pipeline run example

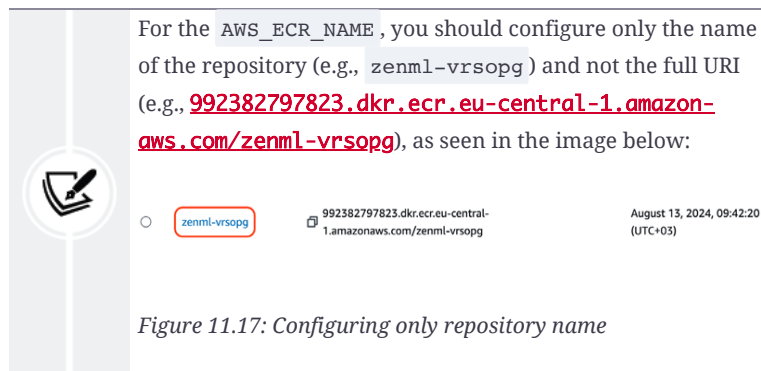
The last step in setting up the CI/CD pipeline is to test it and see how it works.

Test out the CI/CD pipeline

To test the CI/CD pipelines yourself, you must fork the LLM-Engineering repository to have full *write* access to the GitHub repository. Here is the official tutorial on how to fork a GitHub project: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/working-with-forks/fork-a-repo>

The last step is to set up a few secrets that will allow the CD pipeline to log in to AWS and point to the right ECR resource. To do so, go to the **Settings** tab at the top of the forked repository in GitHub. In the left panel, in the **Security** section, click on the **Secrets and Variables** toggle and, finally, on **Actions**. Then, on the **Secrets** tab, create four repository secrets, as shown in *Figure 11.17*. These secrets will be securely stored and accessible only by the GitHub Actions CD pipeline.

The `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` are the AWS credentials you used across the book. In *Chapter 2*, you see how to create them. The `AWS_REGION` (e.g., `eu-central-1`) and `AWS_ECR_NAME` are the same ones used in the **Deploying the LLM Twin's pipelines** to the cloud section.



To trigger the CI pipeline, create a feature branch, modify the code or documentation, and create a PR to the main branch. To trigger the CD pipeline, merge the PR into the main branch.

After the CD GitHub Actions are complete, check the ECR repository to see whether the Docker image was pushed successfully.

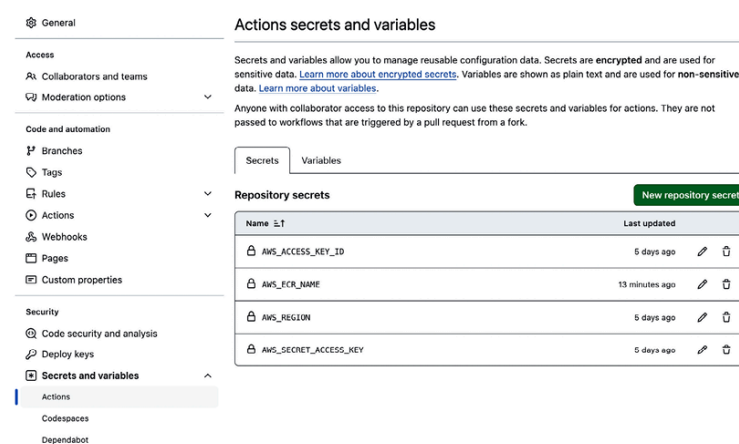


Figure 11.18: GitHub Actions secrets

If you need more details on how to set up GitHub Actions secrets, we recommend checking out their official documentation: <https://docs.github.com/en/actions/security-for-github-actions/security-guides/using-secrets-in-github-actions>

The CT pipeline

To implement the CT pipeline, we will leverage ZenML. Once ZenML (or other orchestrators such as Metaflow, Dagster, or Airflow) orchestrates all your pipelines and your infrastructure is deployed, you are very close to reaching CT.

Remember the core difference between the CI/CD and CT pipelines. The CI/CD pipeline takes care of testing, building, and deploying your code—a dimension that any software program has. The CT pipeline leverages the code managed by the CI/CD pipeline to automate your data, training, and model-serving process, where the data and model dimensions are present only in the AI world.

Before diving into the implementation, we want to highlight two design choices that made reaching CT simple:

- **The FTI architecture:** A modular system with clear interfaces and components made it easy to capture the relationship between the pipelines and automate them.
- **Starting with an orchestrator since day 0:** We started with ZenML at the beginning of the project's development. Early on, we only used it locally. But it acted as an entry point for our pipelines and a way to monitor their execution. Doing so forced us to decouple each pipeline and transfer the communication between them solely through various types of data storage, such as the data warehouse, feature store, or artifact store. As we have leveraged ZenML since day 0, we got rid of implementing a tedious CLI to configure our application. Instead, we did it directly through YAML configuration files out of the box.

In *Figure 11.19*, we can see all the pipelines that we have to chain together to fully automate our training and deployment. The pipelines aren't new; they aggregate everything we've covered throughout this book. Thus, at this point, we will treat them as black boxes that interact with each other.

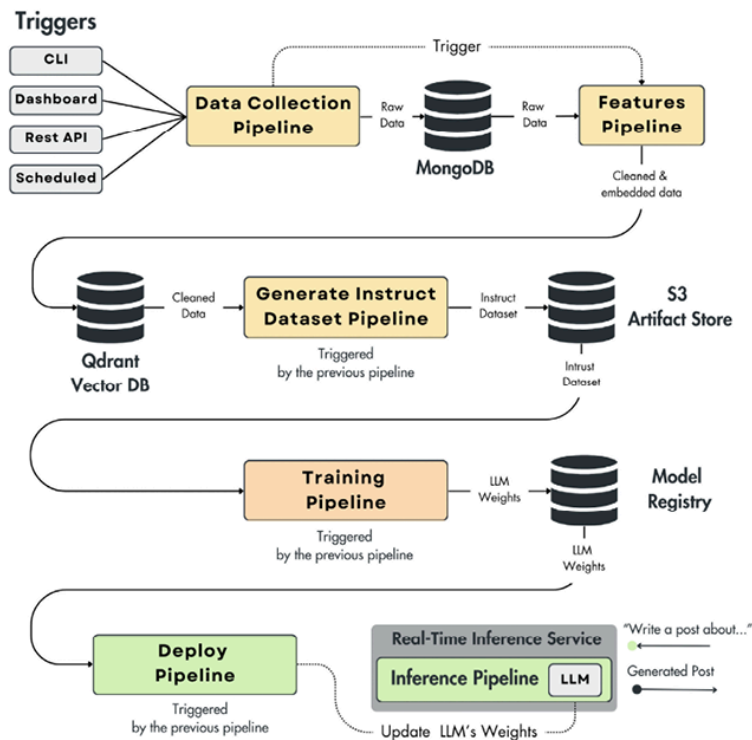


Figure 11.19: CT pipeline

For the LLM Twin's CT pipeline, we have to discuss the initial trigger that starts the pipelines and how the pipelines are triggered by each other.

Initial triggers

As illustrated in Figure 11.18, we initially want to trigger the data collection pipeline. Usually, the triggers can be of three types:

- **Manual triggers:** Done through the CLI or the orchestrator's dashboard, in our case, through the ZenML dashboard. Manual triggers are still extremely powerful tools, as you need just one action to start the whole ML system, from data gathering to deployment, instead of fiddling with dozens of scripts that you might configure wrong or run in an invalid order.
- **REST API triggers:** You can call a pipeline by an HTTP request. This is extremely useful when integrating your ML pipelines with other components. For example, you can have a watcher constantly looking for new articles. It triggers the ML logic using this REST API trigger when it finds some. To find more details on this feature, check out this tutorial on ZenML's documentation:
<https://docs.zenml.io/v/docs/how-to/trigger-pipelines/trigger-a-pipeline-from-rest-api>.
- **Scheduled triggers:** Another common approach is to schedule your pipeline to run constantly on a fixed interval. For example, depending on your use case, you can schedule your pipeline to run daily, hourly, or every minute. Most of the orchestrators, ZenML included, provide a cron expression interface where you can define your execution frequency. In the following example from ZenML, the pipeline is scheduled every hour:

```
Schedule(cron_expression="* * 1 * *")
```


We chose a manual trigger for our LLM Twin use case as we don't have other components to leverage the REST API triggers. Also, as the datasets are generated from a list of static links defined in the ZenML configs, running them on a schedule doesn't make sense as they would always yield the same results.

But a possible next step for the project is to implement a watcher that monitors for new articles. When it finds any, it generates a new config and triggers the pipelines through the REST API. Another option is implementing the watcher as an additional pipeline and leveraging the schedule triggers to look daily for new data. If it finds any, it executes the whole ML system; otherwise, it stops.

The conclusion is that once you can manually trigger all your ML pipelines through a single command, you can quickly adapt it to more advanced and complex scenarios.

Trigger downstream pipelines

To keep things simple, we sequentially chained all the pipelines. More concretely, when the data collection pipeline has finished, it will trigger the feature pipeline. When the feature pipeline has been completed successfully, it triggers the dataset generation pipeline, and so on. You can make the logic more complex, like scheduling the generate instruct dataset pipeline to run daily, checking the amount of new data in the Qdrant vector DB, and starting only if it has enough new data. From this point, you can further tweak the system's parameters and optimize them to reduce costs.

To trigger all the pipelines in one go, we created one master pipeline that aggregates everything in one entry point:

```
@pipeline
def end_to_end_data(
    author_links: list[dict[str, str | list[str]]], ... # Other paramaters...
) -> None:
    wait_for_ids = []
    for author_data in author_links:
        last_step_invocation_id = digital_data_etl(
            user_full_name=author_data["user_full_name"], links=author_data["links"]
        )
        wait_for_ids.append(last_step_invocation_id)
    author_full_names = [author_data["user_full_name"] for author_data in author_links]
    wait_for_ids = feature_engineering(author_full_names=author_full_names, wait_for_ids=wait_for_ids)
    generate_instruct_datasets(...)
    training(...)
    deploy(...)
```

To keep the function light, we added all the logic up to computing the features. But, as we suggested in the code snippet above, you can easily add the instruction dataset generation, training, and deploy logic to the parent pipeline to implement an end-to-end flow. By doing that, you can automate everything from data collection to deploying the model.

To run the end-to-end pipeline, use the following `poe` command:

```
poetry poe run-end-to-end-data-pipeline
```

What we implemented is not the best approach, as it compresses all the steps into a single monolith pipeline (which we want to avoid), as illustrated in *Figure 11.20*. Usually, you want to keep each pipeline isolated and use triggers to start downstream pipelines. This makes the system easier to understand, debug, and monitor.

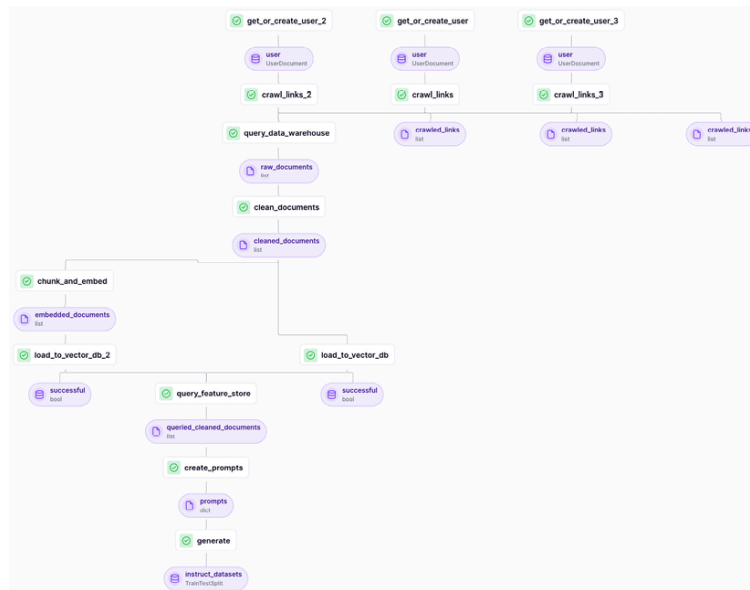


Figure 11.20: End-to-end pipeline illustrated in ZenML's dashboard

Unfortunately, the ZenML cloud's free trial has a limitation of a maximum of three pipelines. As we have more, we avoided that limitation by compressing all the steps into a single pipeline. But if you plan to host ZenML yourself or buy their license, they offer the possibility to independently trigger a pipeline from another pipeline, as you can see in the code snippet below where we triggered the feature engineering pipeline after the data collection ETL:

```
from zenml import pipeline, step
@pipeline
def digital_data_etl(user_full_name: str, links: list[str]) -> str:
    user = get_or_create_user(user_full_name)
    crawl_links(user=user, links=links)
    trigger_feature_engineering_pipeline(user)
@step
def trigger_feature_engineering_pipeline(user):
    run_config = PipelineRunConfiguration(...)
    Client().trigger_pipeline("feature_engineering", run_configuration=run_config)
@pipeline
def feature_engineering(author_full_names: list[str]) -> list[str]:
    ... # ZenML steps
```

By taking this approach, each pipeline will have its independent run, where one pipeline sequentially triggers the next one, as described at the beginning of this section. Note that this feature is not unique to ZenML but is common in orchestrator tools. The principles we have learned so far hold. Only how we interact with the tool changes.

Prompt monitoring

We will use Opik (from Comet ML) to monitor our prompts. But remember from the *LLMOps* section earlier in this chapter that we are not interested only in the input prompt and generated answer.

We want to log the entire trace from the user's input until the final result is available. Before diving into the LLM Twin use case, let's look at a simpler example:

```
from opik import track
import openai
from opik.integrations.openai import track_openai
openai_client = track_openai(openai.OpenAI())
@track
def preprocess_input(text: str) -> str:
    return text.strip().lower()
@track
def generate_response(prompt: str) -> str:
    response = openai_client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=[{"role": "user", "content": prompt}]
    )
    return response.choices[0].message.content
@track
def postprocess_output(response: str) -> str:
    return response.capitalize()
@track(name="llm_chain")
def llm_chain(input_text: str) -> str:
    preprocessed = preprocess_input(input_text)
    generated = generate_response(preprocessed)
    postprocessed = postprocess_output(generated)
    return postprocessed
result = llm_chain("Hello, do you enjoy reading the book?")
```

The preceding code snippet reflects in a simplistic way what most LLM applications will look like. You have the `llm_chain()` main function, which takes the initial input as a parameter and returns the final result.

Then, you have preprocessing and postprocessing functions surrounding the actual LLM call. Using the `@track()` decorator, we log the input and output of each function, which will ultimately be aggregated into a single trace. By doing so, we will have access to the initial input text, the generated answer, and all the intermediary steps required to debug any potential issues using Opik's dashboard.

The last step is to attach the necessary metadata for your use case to the current trace. As seen in the following code snippet, you can easily do that by calling the `update()` method, where you can tag your trace or add any other metadata, such as the number of input tokens, through a Python dictionary:

```
from opik import track, opik_context
@track
def llm_chain(input_text):
    # LLM chain code
    # ...
    opik_context.update_current_trace(
        tags=["inference_pipeline"],
        metadata={
            "num_tokens": compute_num_tokens(...)
        },
    ),
```

```

feedback_scores=[
{
    "name": "user_feedback",
    "value": 1.0,
    "reason": "The response was valuable and correct."
},
{
    "name": "llm_judge_score",
    "value": compute_llm_judge_score(...),
    "reason": "Computing runtime metrics using an LLM Judge."
}
)

```

You can expand on this idea and log various feedback scores. The most common is asking the user if the generated answer is valuable and correct. Another option is to compute various metrics automatically through heuristics or LLM judges.

Finally, let's see how to add prompt monitoring to our LLM Twin project. First, look at *Figure 11.21* and remember our model-serving architecture. We have two microservices, the LLM and business microservices. The LLM microservice has a narrow scope, as it only takes as input a prompt that already contains the user's input and context and returns an answer that is usually post-processed. Thus, the business microservice is the right place to implement the monitoring pipeline, as it coordinates the end-to-end flow. More concretely, Opik implementation will be in the FastAPI server developed in *Chapter 10*.

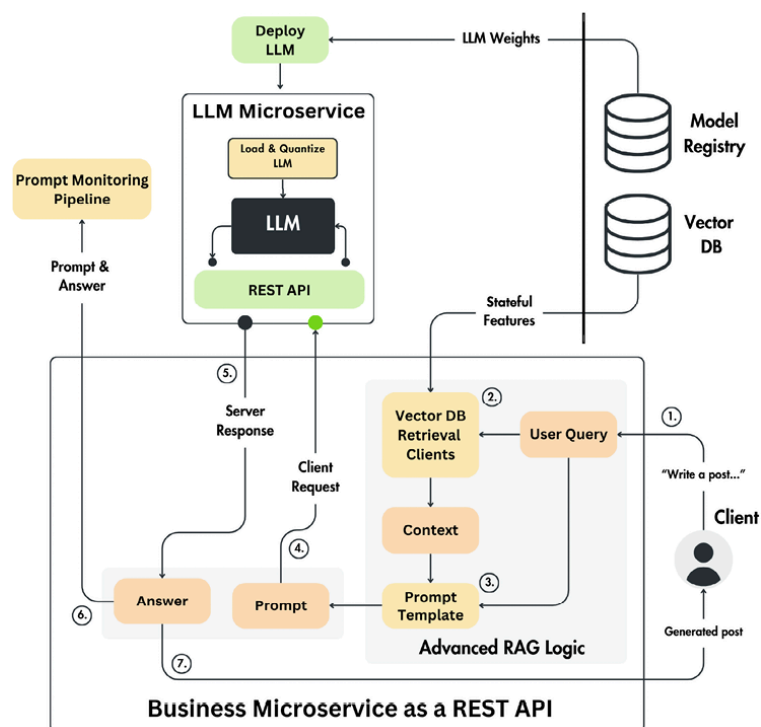


Figure 11.21: Inference pipeline serving architecture

As our implementation is already modular, using Opik makes it straightforward to log an end-to-end trace of a user's request:

```

from opik import track
@track
def call_llm_service(query: str, context: str | None) -> str:

```

```

llm = LLMInferenceSagemakerEndpoint(...)
answer = InferenceExecutor(llm, query, context).execute()
return answer

@track
def rag(query: str) -> str:
    retriever = ContextRetriever()
    documents = retriever.search(query, k=3 * 3)
    context = EmbeddedChunk.to_context(documents)
    answer = call_llm_service(query, context)
    return answer

```

The `rag()` function represents your application's entry point. All the other processing steps take place in the `ContextRetriever` and `InferenceExecutor` classes. Also, by decorating the `call_llm_service()` function, we can clearly capture the prompt sent to the LLM and its response.

To add more granularity to our trace, we can further decorate other functions containing pre- or post-processing steps, such as the `ContextRetriever` search function:

```

class ContextRetriever:
    ...

    @track
    def search(
        self,
        query: str,
        k: int = 3,
        expand_to_n_queries: int = 3,
    ) -> list:
        query_model = Query.from_str(query)
        query_model = self._metadata_extractor.generate(query_model)
        ... # Rest of the implementation

```

Or even go further to the retrieval optimization methods, such as the self-query metadata extractor, to add more granularity:

```

class SelfQuery:

    @track
    def generate(self, query: str) -> str:
        ...
        return enhanced_query

```

The developer is responsible for deciding how much granularity the application needs for proper debugging and analysis. As having detailed monitoring is healthy, monitoring everything can be dangerous as it adds too much noise and makes manually understanding the traces difficult. You must find the right balance. A good rule of thumb is tracing the most critical functions, such as `rag()` and `call_llm_service()`, and gradually adding more granularity when needed.

The last step is to attach valuable metadata and tags to our traces. To do so, we will further enhance the `rag()` function as follows:

```

@track
def rag(query: str) -> str:
    retriever = ContextRetriever()
    documents = retriever.search(query, k=3 * 3)
    context = EmbeddedChunk.to_context(documents)
    answer, prompt = call_llm_service(query, context)
    trace = get_current_trace()
    trace.update(
        tags=["rag"],
        metadata={
            "model_id": settings.HF_MODEL_ID,
            "embedding_model_id": settings.TEXT_EMBEDDING_MODEL_ID,
            "temperature": settings.TEMPERATURE_INFERENCE,
            "prompt_tokens": compute_num_tokens(prompt),
            "total_tokens": compute_num_tokens(answer),
        }
    )
    return answer

```

There are three main aspects that we should constantly monitor:

- **Model configuration:** Here, we should consider both the LLM and other models used within the RAG layer. The most critical aspects of logging are the model IDs, but you can also capture other important information that significantly impacts the generation, such as the temperature.
- **Total number of tokens:** It's critical to constantly analyze the statistics of the number of tokens generated by your input prompts and total tokens, as this significantly impacts your serving costs. For example, if the average of the total number of tokens generated suddenly increases, it's a strong signal that you have a bug in your system that you should investigate.
- **The duration of each step:** Tracking the duration of each step within your trace is essential to finding bottlenecks within your system. If the latency of a specific request is abnormally large, you quickly have access to a report that helps you find the source of the problem.

Alerting

Using ZenML, you can quickly implement an alerting system on any platform of your liking, such as email, Discord, or Slack. For example, you can add a callback in your training pipeline to trigger a notification when the pipeline fails or the training has finished successfully:

```

from zenml import get_pipeline_context, pipeline
@pipeline(on_failure=notify_on_failure)
def training_pipeline(...):
    ...
    notify_on_success()

```

Implementing the notification functions is straightforward. As seen in the code snippets below, you have to get the `alerter` instance from your current stack, build the message as you see fit, and send it to your notification channel of choice:

```
from zenml.client import Client
alerter = Client().active_stack.alerter
def notify_on_failure() -> None:
    alerter.post(message=build_message(status="failed"))
@step(enable_cache=False)
def notify_on_success() -> None:
    alerter.post(message=build_message(status="succeeded"))
```

ZenML and most orchestrators simplify implementing an `alerter`, as it's a critical component in your MLOps/LLMOps infrastructure.

Summary

In this chapter, we laid down the foundations with a theoretical section on DevOps. Then, we moved on to MLOps and its core components and principles. Finally, we presented how LLMOps differs from MLOps by introducing strategies such as prompt monitoring, guardrails, and human-in-the-loop feedback. Also, we briefly discussed why most companies would avoid training LLMs from scratch but choose to optimize them for their use case through prompt engineering or fine-tuning. At the end of the theoretical portion of the chapter, we learned what a CI/CD/CT pipeline is, the three core dimensions of an ML application (code, data, model), and that, after deployment, it is more critical than ever to implement a monitoring and alerting layer due to model degradation.

Next, we learned how to deploy the LLM Twin's pipeline to the cloud. We understood the infrastructure and went step by step through deploying MongoDB, Qdrant, the ZenML cloud, and all the necessary AWS resources to sustain the application. Finally, we learned how to Dockerize our application and push our Docker image to AWS ECR, which will be used to execute the application on top of AWS SageMaker.

The final step was to add LLMOps to our LLM Twin project. We began by implementing a CI/CD pipeline with GitHub Actions. Then, we looked at our CT strategy by leveraging ZenML.

Finally, we saw how to implement a monitoring pipeline using Opik from Comet ML and an alerting system using ZenML. These are the fundamental pillars in adding MLOps and LLMOps to any LLM-based application.

The framework we learned about throughout the book can quickly be extrapolated to other LLM applications. Even if we used the LLM Twin use case as an example, most of the strategies applied can be adapted to other projects. Thus, we can get an entirely new application by changing the data and making minor tweaks to the code. Data is the new oil, remember?

By finalizing this chapter, we've learned to build an end-to-end LLM application, starting with data collection and fine-tuning until deploying the LLM microservice and RAG service. Throughout this book, we aimed to provide a thought framework to help you build and solve real-world problems in the GenAI landscape. Now that you have it, we wish you good luck in your journey and happy building!

References

- GitLab. (2023, January 25). *What is DevOps?* | *GitLab*. GitLab.
<https://about.gitlab.com/topics/devops/>
- Huyen, C. (2024, July 25). Building a generative AI platform. *Chip Huyen*. <https://huyenchip.com/2024/07/25/genai-platform.html>
- *Lightricks customer story: Building a recommendation engine from scratch*. (n.d.). <https://www.qwak.com/academy/lightricks-customer-story-building-a-recommendation-engine-from-scratch>
- *What LLMOps*. (n.d.). Google Cloud.
<https://cloud.google.com/discover/what-is-llmops?hl=en>
- *MLOps: Continuous delivery and automation pipelines in machine learning*. (2024, August 28). Google Cloud.
https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning#top_of_page
- *Ml-ops.org*. (2024a, July 5). <https://ml-ops.org/content/mlops-principles>
- *Ml-ops.org*. (2024b, July 5). <https://ml-ops.org/content/mlops-principles>
- *Ml-ops.org*. (2024c, July 5). <https://ml-ops.org/content/motivation>
- Mohandas, G. M. (2022a). Monitoring machine learning systems. *Made With ML*. <https://madewithml.com/courses/mlops/monitoring/>
- Mohandas, G. M. (2022b). Testing Machine Learning Systems: Code, Data and Models. *Made With ML*. <https://madewithml.com/courses/mlops/testing/>
- Preston-Werner, T. (n.d.). *Semantic Versioning 2.0.0*. Semantic Versioning. <https://semver.org/>
- Ribeiro, M. T., Wu, T., Guestrin, C., & Singh, S. (2020, May 8). *Beyond Accuracy: Behavioral Testing of NLP models with CheckList*. arXiv.org.
<https://arxiv.org/abs/2005.04118>
- Wandb. (2023, November 30). *Understanding LLMOps: Large Language Model Operations*. Weights & Biases. <https://wandb.ai/site/articles/understanding-llmops-large-language-model-operations/>
- Zenml-Io. (n.d.). *GitHub—zenml-io/zenml-huggingface-sagemaker: An example MLOps overview of ZenML pipelines from a Hugging Face model repository to a deployed AWS SageMaker endpoint*. GitHub.
<https://github.com/zenml-io/zenml-huggingface-sagemaker/tree/main>

Unlock this book's exclusive benefits
now

This book comes with additional benefits
designed to elevate your learning
experience.

<https://www.packtpub.com/unlock/9781836200079>

*Note: Have your purchase invoice ready
before you begin.*

Answers collapsed